

Coding Task 1

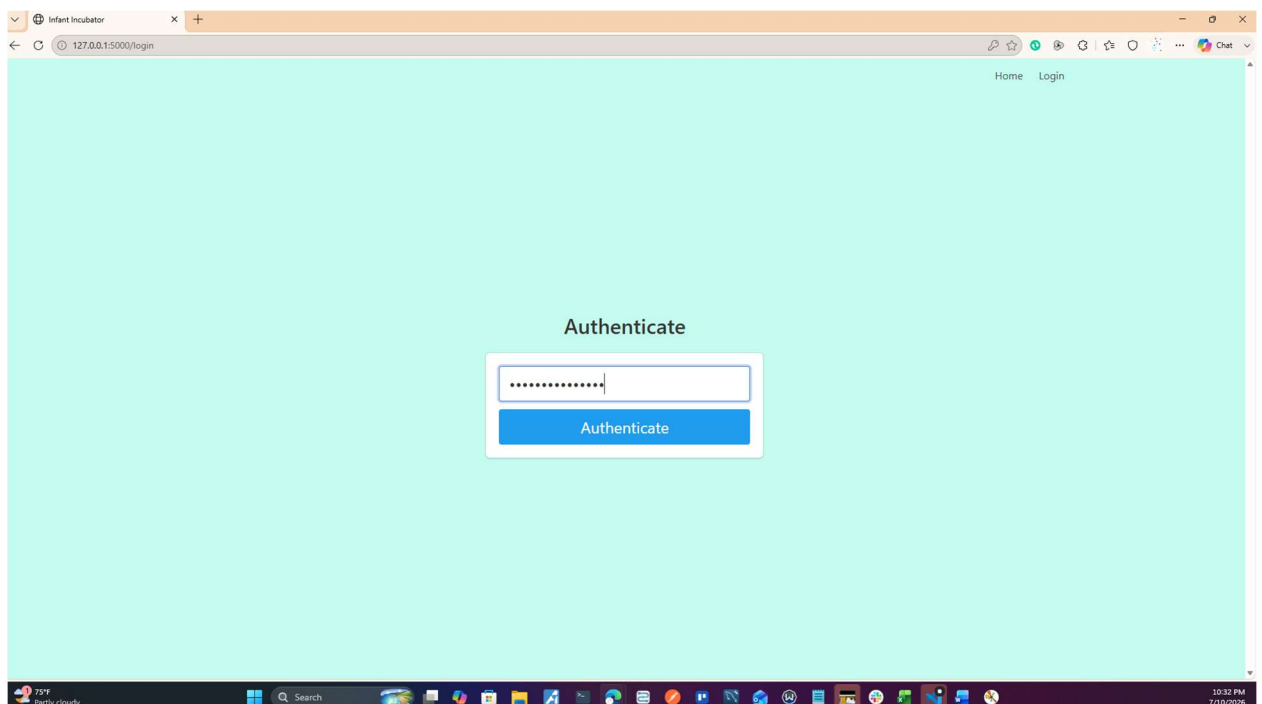
1. Find the SQL Injection of the login page. You need to submit a screenshot of the webpage after having logged in using the payload you find.

The vulnerable code is in `app.py` inside the login handler `def login():`

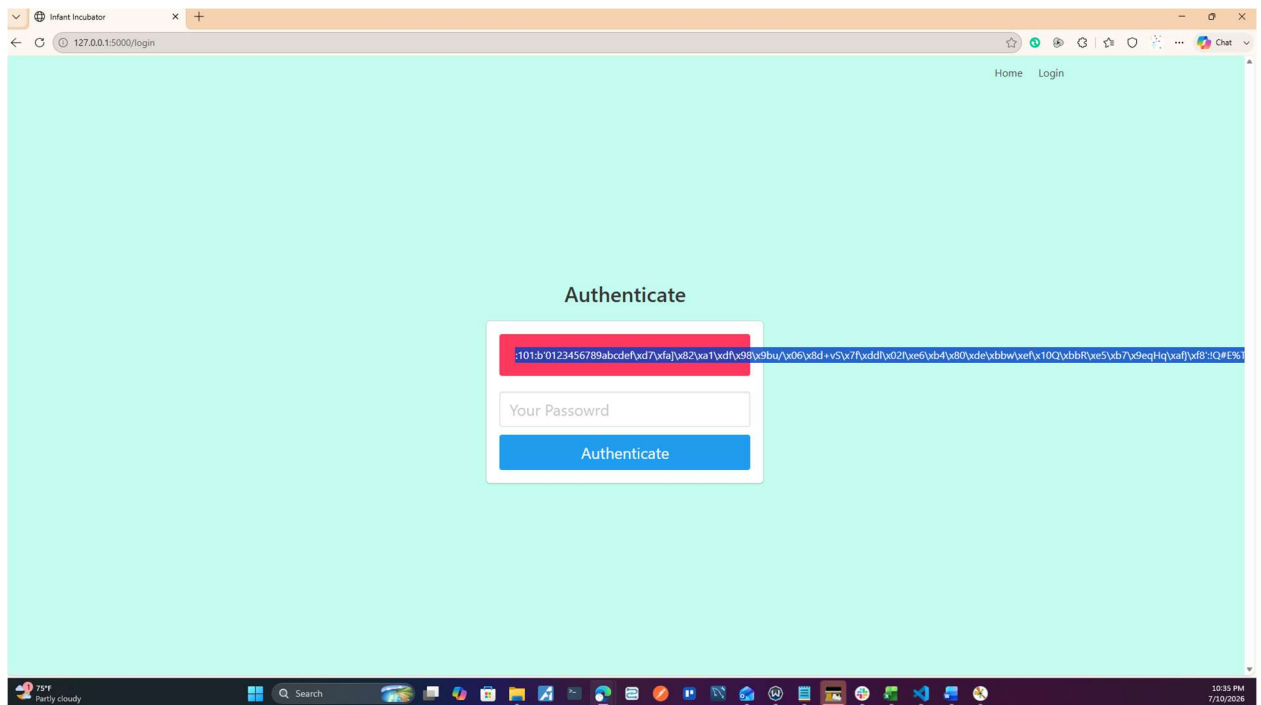
- `app.py` : `line 59` builds a SQL query by concatenating user input directly into the string:

- `db_query = "SELECT * FROM users WHERE password = " + password + ""`

That means the login form field named `authToken` is the injection point.



I used the payload '**OR '1'='1'**' –



That output is the app echoing the database row values from the vulnerable query, which confirms the injection worked. The page is showing the values from the database row that the vulnerable query returned. In other words:

- 101 = the user ID
- the long blob = the encrypted password stored in the database
- !Q#E%T&U8i6y4r2w = the activation token

So the payload successfully altered the SQL query and caused the app to reveal the row contents.

2. Retrieve the plaintext password and submit the same.

The plaintext password was recovered by decrypting the stored password value using the decrypt script in `decrypt.py`.

The recovered plaintext password is: `SQLi_is_easier_than_this`

3. Please submit the code you use to decrypt the password and name it `decrypt.py`

```
import sqlite3
```

```
from pathlib import Path
```

```
from Crypto.Cipher import AES
```

```
KEY = bytes(
```

```
b"\x93n\x12\xcbC\xe0|\xd0\xa6%7(?KW\xa9\xc2\x02\x97\xc6\\\xd6\xd9c\xf4\xb9\xe2\x89  
\x88<\x9d"
```

```
)
```

```
DB_PATH = Path(__file__).resolve().parent / "database.db"
```

```
def decrypt_password(db_path: Path = DB_PATH) -> str:
```

```
    conn = sqlite3.connect(str(db_path))
```

```
    cur = conn.cursor()
```

```
    row = cur.execute("SELECT * FROM users").fetchone()
```

```
    conn.close()
```

```
    if row is None:
```

```
        raise ValueError("No users found in database")
```

```
    encrypted_password = row[1]
```

```
    if isinstance(encrypted_password, str):
```

```
        encrypted_password = encrypted_password.encode("latin-1")
```

```
    nonce = encrypted_password[:16]
```

```
    tag = encrypted_password[16:32]
```

```
    ciphertext = encrypted_password[32:]
```

```
cipher = AES.new(KEY, AES.MODE_EAX, nonce=nonce)
plaintext = cipher.decrypt_and_verify(ciphertext, tag)
return plaintext.decode("utf-8")
```

```
if __name__ == "__main__":
    print("Database:", DB_PATH)
    print("Plaintext password:", decrypt_password(DB_PATH))
```

4. After the above tasks, you have to find at least 5 new vulnerabilities in the code.

Vulnerability 1: **Information disclosure through verbose error handling**

- In `login()`, the code appends raw values from the database row into `db_message` and renders them back to the page through `Err=str(db_message)`.
- This leaks internal database data and row structure to the client.
- Fix: return a generic message such as "Login failed" and log the detailed error on the server side only.

Vulnerability 2: **Missing CSRF protection on state-changing forms**

- The POST endpoints `/login`, `/get_temp`, `/set_temp_c`, `/set_temp_f`, and `/set_temp_k` accept form submissions without CSRF tokens or server-side validation.
- Fix: enable Flask-WTF CSRF protection or add a CSRF token to each form and verify it before processing the request.

Vulnerability 3: **Weak command parsing in the network server**

- `SampleNetworkServer.py` accepts any text message and sends it into `processCommands()` after a simple token check.

- A malformed or unexpected message can trigger unintended behavior because the parser does not enforce a strict command grammar.
- Fix: define a strict command schema, reject unknown commands, and enforce explicit message boundaries.

Vulnerability 4: `run()` / `processCommands()` interaction issue in `SampleNetworkServer.py`

- In `SampleNetworkServer.py`, `run()` decides whether the request is protected or open and then forwards the payload to `processCommands()`.
- Because the same dispatcher handles both open and authenticated traffic, the flow can execute commands in the wrong context and lacks a clear separation between initial authentication and later protected commands.
- Fix: split the handler into two explicit paths: one for `AUTH` / `LOGOUT` and one for protected commands, and reject open commands once the session has entered the protected mode.

Vulnerability 5: **Inconsistent schema and hidden data storage**

- `schema.sql` defines the `users` table with only `user\_id` and `password`, but the live `database.db` contains an additional `act\_token` value.
- This mismatch can cause inconsistent behavior, hidden data retention, and confusion during debugging or maintenance.
- Fix: align the schema file with the actual database schema and enforce migrations instead of relying on manual database edits.

5. Additional notes

- The DB schema in `schema.sql` does not include `act\_token`, but the actual `database.db` contains it. This suggests schema inconsistency and hidden data storage.
- The application leaks sensitive data back to the user through error rendering.
- The login page should never echo raw database row values in an error message.

